

LegalEase - Sample Project Documentation

[illegible]

Maximum Marks: Project Documentation

1. Project Overview

LegalEase is an AI-powered legal document generator built with Python. It uses a Streamlit frontend, FastAPI backend and Google Gemini API to generate draft legal documents from structured user input.

2. Problem Statement

Preparing routine legal documents can be repetitive and time-consuming. Users may also struggle with legal wording and document formatting.

3. Proposed Solution

LegalEase provides a guided form for document type, parties, terms and effective date, then generates a draft through Gemini and supports review, editing and export.

4. Target Users

Students, freelancers, small businesses, service providers and other users needing initial drafts.

5. User Workflow

Enter details → Generate → Preview → Edit if needed → Download.

6. AI Integration

The Gemini generator constructs a prompt from the user inputs and requests generated document text from Google's Gemini API.

7. Frontend

Streamlit provides the input form, generation button, preview card, edit control and three download buttons.

8. Backend

FastAPI exposes POST /generate and GET /health. Pydantic validates the request body.

9. Document Formatting

DOCX is created using python-docx, PDF using fpdf2, and HTML is used for in-app preview.

10. Configuration

API credentials and server settings are read from .env/environment configuration.

11. Security Considerations

Secrets are not intended to be stored in source code. Users should avoid entering unnecessary sensitive personal information and should review generated documents.

12. Error Handling

The backend distinguishes configuration, upstream API and unexpected failures. The frontend shows warnings for missing fields and connection/API failures.

13. Functional Features

Eight functional features are documented: structured input, AI generation, preview, editing, TXT export, DOCX export, PDF export, and health/error handling.

14. Testing

Manual functional scenarios cover health, generation, validation, backend connection handling and exports. Formal load testing remains a future task.

15. Code Organization

The codebase is divided into `ai_core`, `legalEaseAPI`, `frontend`, `formatting` and `utils` modules.

16. Installation

Python environment creation, dependency installation, `.env` configuration and separate backend/frontend commands are documented in `README.md`.

17. Deployment

The backend can be hosted on a Python-capable service and the frontend on Streamlit Community Cloud or a suitable host. Deployment is not included in the submitted source.

18. Limitations

AI output can require legal review; external API availability affects generation; current model names may need updating as providers retire or rename models.

19. Future Enhancements

Add authentication, document history, templates, clause explanation, automated tests, logging, deployment and stronger legal-domain validation.

20. Demonstration Scenario

Demonstrate creating an NDA or service agreement, previewing the result, editing a clause, and downloading DOCX/PDF.

21. Conclusion

LegalEase demonstrates an end-to-end Generative AI application with a practical user workflow, API integration, modular Python architecture and document export capabilities.